Universidade Federal Fluminense — IC/UFF

Computação Gráfica — Prof. Marcos Lage

Relatório Técnico: Implementação de Cena 2D de Pipa em WebGPU

Felipe Cabral Liporage **Eduardo Rottschaefer Oliveira** **Allan Vignoli dos Santos**

12 de outubro de 2026

Código-fonte: https://github.com/Rottschaefer/webgpu-kite-2d

1 Introdução

O trabalho consiste em uma cena 2D feita em **WebGPU**, com shaders em **WGSL**, mostrando uma pipa voando sobre um céu azul de cor sólida. O código foi construído a partir dos exemplos feitos em sala, reaproveitando os arquivos:

- **`lib/webgpu.js`:** cuida da parte de configuração da WebGPU, como pegar o *device*, configurar o canvas, criar o pipeline e os buffers.
- **`lib/utils.js`:** funções para criar as matrizes 4×4 que usamos (translação, rotação, câmera e projeção ortográfica).
- **Shaders:** o vertex shader (`projection.vert.js`) multiplica cada vértice pela matriz final recebida no *uniform buffer*, e o fragment shader (`projection.frag.js`) só pinta com a cor do vértice.

A animação roda chamando `requestAnimationFrame` a cada quadro, que atualiza o tempo e desenha a cena de novo.

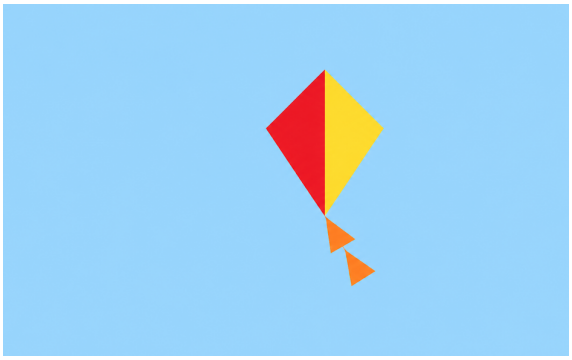


Figura 1: Cena da pipa rodando no navegador.

2 Modelagem da Pipa

A cena usa apenas quatro triângulos, todos no plano $Z = 0$:

- **Pipa:** dois triângulos, um vermelho e um amarelo, que dividem a aresta do meio.
- **Cauda:** duas fitas laranjas, uma presa embaixo da outra. Cada fita foi desenhada com a ponta de cima em $(0, 0)$.

Como a pipa e cada fita se movem de forma diferente, separamos os vértices em três *vertex buffers*: um para a pipa (6 vértices) e um para cada fita (3 vértices cada). O céu é só a cor de fundo usada para limpar a tela (`clearColor`) antes de desenhar.

Componente	**Cor RGB**	**Vértices Locais** (x, y)	**Qtd.**
Pipa (Metade Esquerda)	$(0.89, 0.12, 0.12)$	$(0.0, 0.55), (-0.28, 0.25), (0.0, -0.20)$	3
Pipa (Metade Direita)	$(0.99, 0.80, 0.08)$	$(0.0, 0.55), (0.0, -0.20), (0.28, 0.25)$	3
Fita 1 (Superior)	$(0.98, 0.46, 0.08)$	$(0.00, 0.00), (0.02, -0.20), (0.14, -0.12)$	3
Fita 2 (Inferior)	$(0.98, 0.46, 0.08)$	$(0.00, 0.00), (0.02, -0.19), (0.14, -0.10)$	3

Tabela 1: Triângulos utilizados na modelagem da cena.

3 Transformações Geométricas e Animação

3.1 Movimento Vertical da Pipa

Para a pipa subir e descer sem parar, usamos um seno do tempo como deslocamento em Y:

$$y(t) = 0.35 \cdot \sin(t) \tag{1}$$

A cada quadro o tempo t aumenta 0.03, então a pipa vai de -0.35 até $+0.35$ e volta, de forma suave. Esse valor vira uma translação $\mathbf{T}(0,\, y(t))$.

3.2 Rotação da Pipa

A rotação em Z sempre gira o objeto em torno da origem. Como a origem fica perto da parte de baixo da pipa, girar direto faria ela balançar torto. Para girar em torno do meio da pipa, levamos o centro da pipa ($y = 0.175$) para a origem, giramos e depois devolvemos para o lugar:

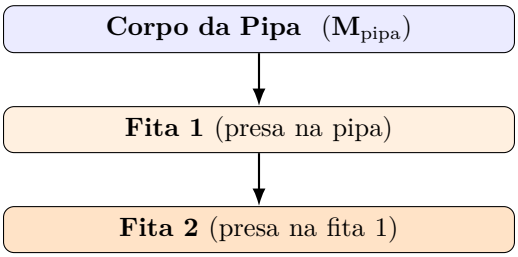
$$\mathbf{M}_{\text{rot}} = \mathbf{T}(0,\, 0.175) \cdot \mathbf{R}_z(\theta) \cdot \mathbf{T}(0,\, -0.175) \tag{2}$$

O ângulo varia com o tempo por $\theta(t) = 0.14 \cdot \sin(0.8\, t)$, ou seja, a pipa inclina cerca de 8° para cada lado. Por fim, a matriz da pipa junta a rotação com a subida e descida da seção anterior:

$$\mathbf{M}_{\text{pipa}} = \mathbf{T}(0,\, y(t)) \cdot \mathbf{M}_{\text{rot}} \tag{3}$$

3.3 Movimento da Cauda

A ideia é que a cauda acompanhe a pipa, mas balance do seu próprio jeito. Para isso, cada fita usa a matriz da peça a que está presa e aplica por cima a sua própria rotação. Assim, quando a pipa se move, as fitas vão junto automaticamente:



- **Fita 1:** é levada até a ponta de baixo da pipa, em $(0, -0.20)$, e gira com $\theta_1(t) = 0.20 \cdot \sin(1.6\, t)$. Ela balança duas vezes mais rápido que a pipa, já que é mais leve.

$$\mathbf{M}_{\text{fita1}} = \mathbf{M}_{\text{pipa}} \cdot \mathbf{T}(0,\, -0.20) \cdot \mathbf{R}_z(\theta_1)$$

- **Fita 2:** é levada até a ponta da fita 1, em $(0.02, -0.20)$, e gira com $\theta_2(t) = 0.28 \cdot \sin(1.6\, t - 0.5)$. O ângulo é um pouco maior e o movimento vem levemente atrasado em relação à fita 1, o que dá o efeito de "chicote" da cauda.

$$\mathbf{M}_{\text{fita2}} = \mathbf{M}_{\text{fita1}} \cdot \mathbf{T}(0.02,\, -0.20) \cdot \mathbf{R}_z(\theta_2)$$

4 Câmera

A câmera é criada em `Scene.createViewMatrix()` usando a função `createLookAtMat4`. Para a câmera subir e descer, basta mudar o Y da posição dela (`eye`) e do ponto para onde ela olha (`at`).

O trabalho pede que a câmera acompanhe a pipa só parcialmente. Se ela seguisse a pipa por completo, a pipa ficaria parada no meio da tela e, como o fundo é de cor sólida, não daria para perceber que ela está se movendo. Por isso, a câmera anda só metade do que a pipa anda:

$$y_{\text{cam}}(t) = 0.5 \cdot y(t) \tag{4}$$

Assim, na tela a pipa ainda sobe e desce, só que com metade da amplitude. Os vetores usados foram:

- `eye` $= [0,\ y_{\text{cam}},\ 2]$: a câmera fica na frente da cena;
- `at` $= [0,\ y_{\text{cam}},\ 0]$: olha reto para a pipa, na mesma altura;
- `up` $= [0,\ 1,\ 0]$.

5 Projeção Ortográfica

A projeção é criada em `Scene.createProjectionMatrix()`, com base no exemplo de aula `07 - projeções`:

```
const aspect = 1024 / 768;
return createOrthographicMat4(-1.6 * aspect, 1.6 * aspect, -1.6, 1.6, 0.1, 10);
```

- **Proporção da tela:** como o canvas é 1024×768 (mais largo que alto), os limites em X são multiplicados pelo `aspect`. Sem isso a pipa apareceria esticada na horizontal.
- **Tamanho da área visível:** o intervalo $[-1.6, 1.6]$ em Y deixa uma folga para que a pipa e a cauda nunca saiam da tela, mesmo nos pontos mais alto e mais baixo do movimento.
- **Near e far:** com `near` $= 0.1$ e `far` $= 10$, a pipa (que está a 2 unidades da câmera) fica dentro do volume visível.

6 Resultado

A cada quadro, o método `Scene.drawShape()` monta a matriz final de cada parte (pipa, fita 1 e fita 2) e envia para a GPU:

$$\mathbf{M}_{\text{final}} = \mathbf{P} \cdot \mathbf{V} \cdot \mathbf{M} \tag{5}$$

onde $\mathbf{M}$ é a matriz de modelagem (posiciona e gira a peça), $\mathbf{V}$ é a matriz da câmera e $\mathbf{P}$ é a projeção ortográfica. Essa matriz é gravada no *uniform buffer* com `writeBuffer`, e o vertex shader multiplica cada vértice por ela.